P2652R0: Disallow user specialization of allocator_traits

Pablo Halpern <phalpern@halpernwightsoftware.com>
2022-10-10 15:39 EDT
Target audience: LEWG/LWG

Abstract

The `allocator_traits` class template was introduced in C++11 with two goals in mind: 1) Provide default implementations for allocator types and operations, thus minimizing the requirements on allocators [allocator.requirements], and 2) provide a mechanism by which future standards could extend the allocator interface without changing allocator requirements and thus obsoleting existing allocators. The latter goal is undermined, however, by the standard currently allowing user-defined specializations of `std::allocator_traits`. Although the standard requires that any such specialization conform to the standard interface, it is not practical to change the standard interface – even by extending it – without breaking any existing user specializations. Indeed, the Sep 2022 C++23 CD, N4919 contains an extension, `allocate_at_least`, that logically belongs in `std::allocator_traits`, but is expressed as an unrelated function because of the problem of potential user-defined specializations.

This paper proposes two possible solutions to this problem: 1) remove the user's latitude for specializing `std::allocator_traits` or 2) deprecate `std::allocator_traits` entirely in favor of an "unbundled" set of allocator-interface functions.

This paper is the proposed resolution to a US NB comment having the same title.

Motivation

The minimal interface for a type conforming to the allocator requirements is that it have a `value_type` type, `allocate` and `deallocate` member functions, and equality comparison operators. The `allocator_traits` class template provides many other types and functions such as `pointer`, `rebind`, and `construct`. Generic types that use allocators are required to access the allocator through `std::allocator_traits`. The latter requirement was intended to allow the allocator interface to be extended without necessarily changing every existing allocator.

For example, C++03 allocators did not have a `void_pointer` member, but such a member is provided automatically through `allocator_traits`; an allocator class can override the default provided by `allocator_traits`, but is not required to do so.

The Standard description for each trait `X` in `std::allocator_traits<A>` typically follows the form, "`a.X` if that expression is well-formed; otherwise *some default*." There is never any reason to specialize `std::allocator_traits` because any trait can be overridden simply by defining the appropriate member within the specific allocator class template. Unfortunately, the standard allows such user specialization and even implies that it is a reasonable thing to do. This allowance prevents the Standards Committee from adding new members to `std::allocator_traits` without breaking existing user specializations.

In P0401R1, `allocate_at_least` was proposed as a static member of `std::allocator_traits` but it was changed to a free function in P0401R2 following a poll in LEWG in Cologne after it was pointed out that, because `std::allocator_traits` can be specialized and that existing specializations would not have the `allocate_at_least` member. It is this free function that is now in the September 2022 C++23 CD, N4919. The current state of affairs, then, is that accessing an allocator is starting to become a hodgepodge of `std::allocator_traits` member-function calls and free-function calls. Before we standardize C++23, we should make an attempt to prevent this divergence.

Proposed resolution 1 of 2

This proposed resolution would disallow user specializations of `std::allocator_traits`. This change would be a breaking one, as existing specializations would become non-conforming. However, with the exception of the new `allocate_at_least` feature, existing code should continue to work for the time being. It is expected that specializations of `std::allocator_traits` are very rare, so the amount of potential breakage should be quite limited.

Wording for PR 1:

Modify section 16.4.4.6.1 [allocator.requirements.general], paragraph 3 as follows:

> The class template `allocator_traits` (20.2.8) supplies a uniform interface to all allocator types. This subclause describes the requirements on allocator types and thus on types used to instantiate `allocator_traits`. A requirement is optional if a default for a given type or expression is specified. Within the standard library `allocator_traits` template, an optional requirement that is not supplied by an allocator is replaced by the specified default type or expression. ~~A user specialization of allocator_traits may provide different defaults and may provide defaults for different requirements than the primary template.~~ If a program declares an explicit or partial specialization of `allocator_traits`, the program is ill-formed, no diagnostic required.

And Paragraph 46 as follows:

> *Remarks*: ~~An allocator need not support `allocate_at_least`, but no default is provided in `allocator_traits`. If an allocator has an `allocate_at_least` member, it shall satisfy the requirements.~~ Default: `{a.allocate(n), n}`.

In section 20.2.2 [memory.syn], remove the non-member declaration for `allocate_at_least`:

```
~~template<class Allocator>~~
~~  [[nodiscard]] constexpr allocation_result<typename~~
~~allocator_traits<Allocator>::pointer>~~
~~    allocate_at_least(Allocator& a, size_t n);                    // freestanding~~
```

In section 20.2.9.1 [allocator.traits.general], add a new member in `allocator_traits`, probably immediately before `deallocate`:

```
template<class Allocator>
  [[nodiscard]] static constexpr allocation_result<pointer>
    allocate_at_least(Allocator& a, size_t n);
```

And in section 20.2.9.3 [allocator.traits.members], add its definition

```
template<class Allocator>
  [[nodiscard]] static constexpr allocation_result<pointer>
    allocate_at_least(Allocator& a, size_t n);
```

> *Returns*: `a.allocate_at_least(n)` if that expression is well-formed; otherwise, `{a.allocate(n), n}`.

Finally, in section 20.2.9.4 [allocator.traits.other] paragraph 2, remove the definition of non-member `allocate_at_least`:

```
~~template<class Allocator>~~
~~  [[nodiscard]] constexpr allocation_result<typename~~
~~allocator_traits<Allocator>::pointer>~~
~~    allocate_at_least(Allocator& a, size_t n);~~
```

~~*Returns*: `a.allocate_at_least(n)` if that expression is well-formed; otherwise, `{a.allocate(n), n}`.~~

Proposed resolution 2 of 2

Another possible resolution is to move *everything* in the direction of `allocate_at_least`, i.e., deprecate `allocator_traits` and replace it by a set of namespace-scoped alias templates and free function templates. Such a change is too large for C++23, but if we can agree on this direction now, it can be made as a non-breaking change in C++26.

Complete proposed wording is not supplied herein, but the general idea is that each type in `allocator_traits` would be replaced by a type trait or free function, similar to the following:

```
template <class Alloc> using allocator_pointer = see below ;
```

Type: `Alloc::pointer` if the qualified-id `Alloc::pointer` is valid and denotes a type (13.10.3); otherwise, `allocator_value_type<Alloc>*`.

```
template <class Alloc, class T>
  constexpr void allocator_destroy(Alloc& a, T* p);
```

Effects: Calls `a.destroy(p)` if that call is well-formed; otherwise, invokes `destroy_at(p)`.

Repeat for each trait in `allocator_traits` change the description of how allocator-aware containers use allocators to use these new versions. The existing `allocator_traits` template would be moved into Appendix D and each member would be specified in terms of the new traits.

Conclusion

The status quo would have allocator operations specified as a mixture of `allocator_traits` members and namespace-scope traits. We should decide which is our preferred method of specification. If we prefer a set of loosely related namespace-scope traits, then nothing needs to be done today except deciding this direction for the future. If, however, we prefer to use `allocator_traits`, then user specializations must be disallowed in C++23, *before* the standard starts drifting in the other direction.

References

N4919: Programming Languages – C++, Committee Draft, 2022-09-05.

P0401R6: Providing size feedback in the Allocator interface, Jonathan Wakely and Chris Kennelly, 2021-01-22